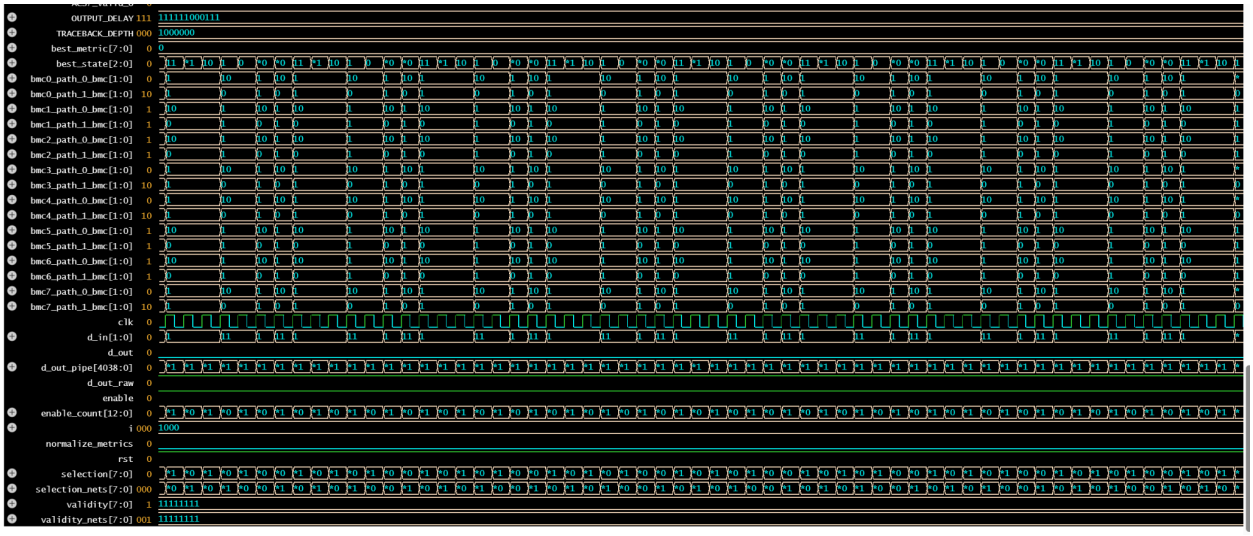
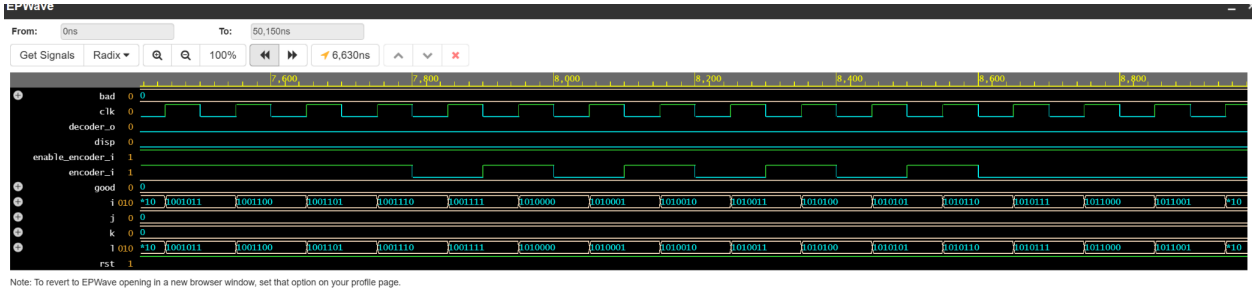
David Brin Term Project

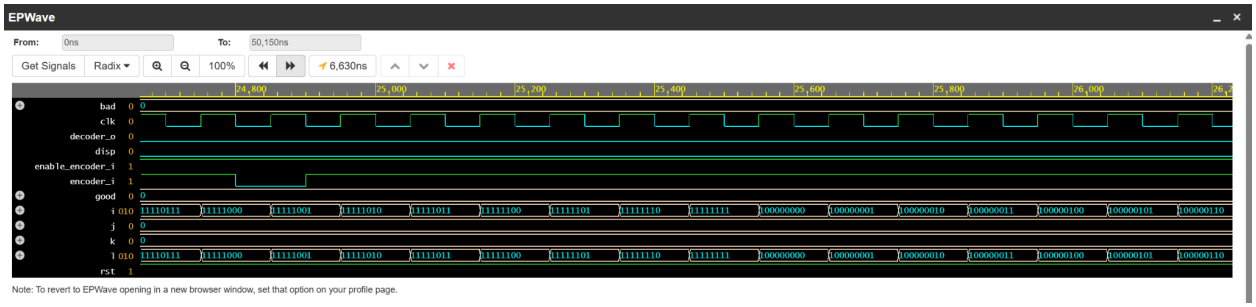
Part 1

```
# yaa! in = 1, out = 1, w_ct =         226, err = 00
# yaa! in = 1, out = 1, w_ct =         227, err = 00
# yaa! in = 1, out = 1, w_ct =         228, err = 00
# yaa! in = 1, out = 1, w_ct =         229, err = 00
# yaa! in = 1, out = 1, w_ct =         230, err = 00
# yaa! in = 1, out = 1, w_ct =         231, err = 00
# yaa! in = 1, out = 1, w_ct =         232, err = 00
# yaa! in = 1, out = 1, w_ct =         233, err = 00
# yaa! in = 1, out = 1, w_ct =         234, err = 00
# yaa! in = 1, out = 1, w_ct =         235, err = 00
# yaa! in = 1, out = 1, w_ct =         236, err = 00
# yaa! in = 1, out = 1, w_ct =         237, err = 00
# yaa! in = 1, out = 1, w_ct =         238, err = 00
# yaa! in = 1, out = 1, w_ct =         239, err = 00
# yaa! in = 1, out = 1, w_ct =         240, err = 00
# yaa! in = 1, out = 1, w_ct =         241, err = 00
# yaa! in = 1, out = 1, w_ct =         242, err = 00
# yaa! in = 1, out = 1, w_ct =         243, err = 00
# yaa! in = 1, out = 1, w_ct =         244, err = 00
# yaa! in = 1, out = 1, w_ct =         245, err = 00
# yaa! in = 1, out = 1, w_ct =         246, err = 00
# yaa! in = 1, out = 1, w_ct =         247, err = 00
# yaa! in = 0, out = 0, w_ct =         248, err = 00
# yaa! in = 1, out = 1, w_ct =         249, err = 00
# yaa! in = 1, out = 1, w_ct =         250, err = 00
# yaa! in = 1, out = 1, w_ct =         251, err = 00
# yaa! in = 1, out = 1, w_ct =         252, err = 00
# yaa! in = 1, out = 1, w_ct =         253, err = 00
# yaa! in = 1, out = 1, w_ct =         254, err = 00
# yaa! in = 1, out = 1, w_ct =         255, err = 00
# corrupted_bits =           0, OUT: good =         256, bad =           0
# ** Note: $stop    : testbench.sv(91)
#    Time: 1045 us  Iteration: 0  Instance: /viterbi_tx_rx_tb
# Break in Module viterbi_tx_rx_tb at testbench.sv line 91
# Stopped at testbench.sv line 91
# exit
```

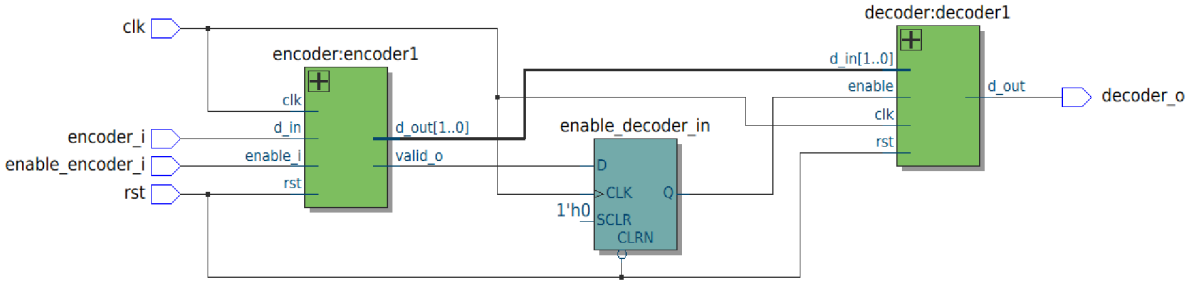
	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	485
2		
3	▼ Combinational ALUT usage for logic	401
1	-- 7 input functions	4
2	-- 6 input functions	60
3	-- 5 input functions	19
4	-- 4 input functions	51
5	-- <=3 input functions	267
4		
5	Dedicated logic registers	642
6		
7	I/O pins	5
8	Total MLAB memory bits	0
9	Total block memory bits	4038
10		
11	Total DSP Blocks	0
12		
13	Maximum fan-out node	clk...put
14	Maximum fan-out	643
15	Total fan-out	5361
16	Average fan-out	5.09

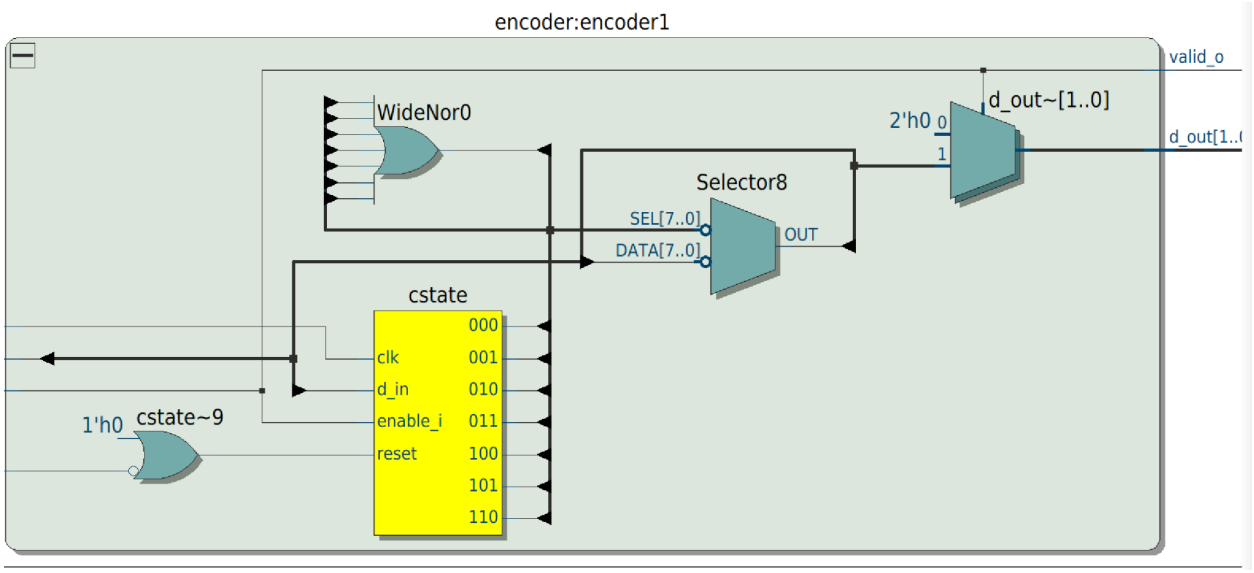


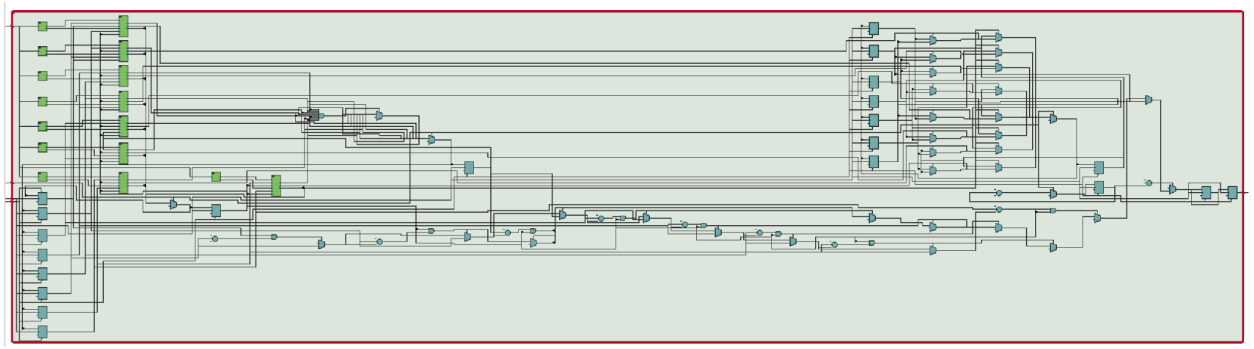


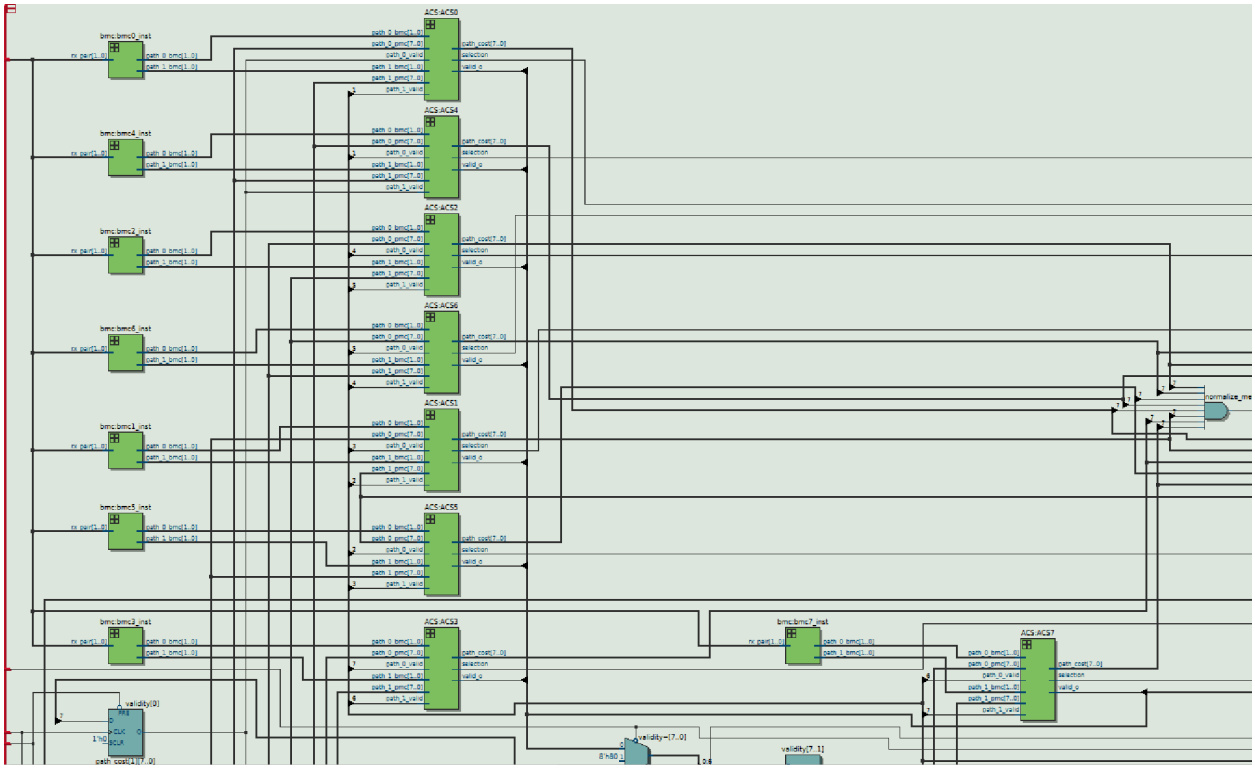


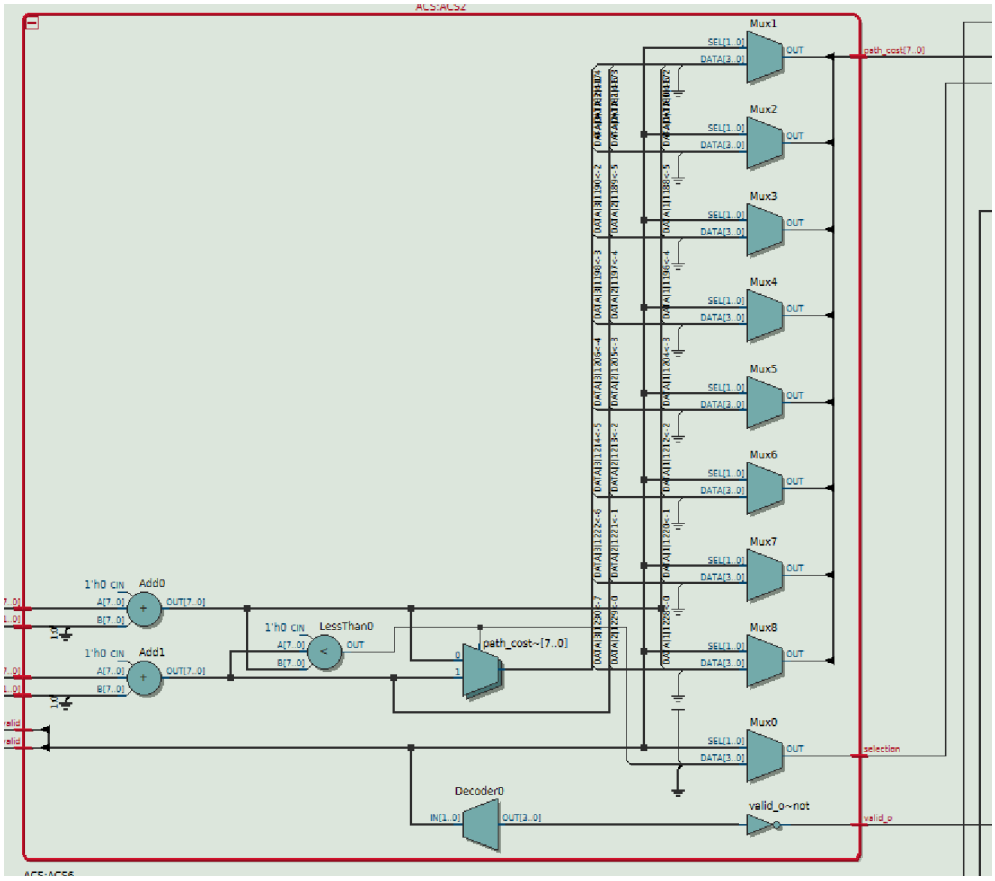
(path cost ACS0, 6, 7 towards the end but output was truncated because EDA wave couldn't load more)

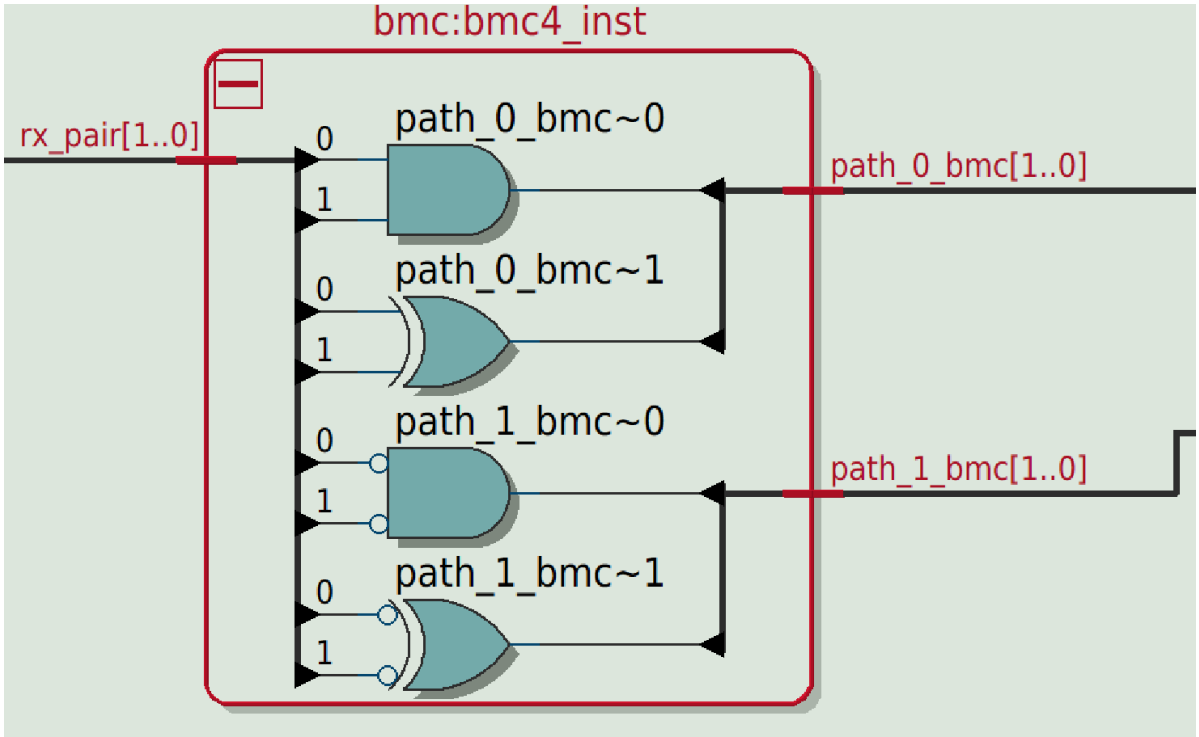


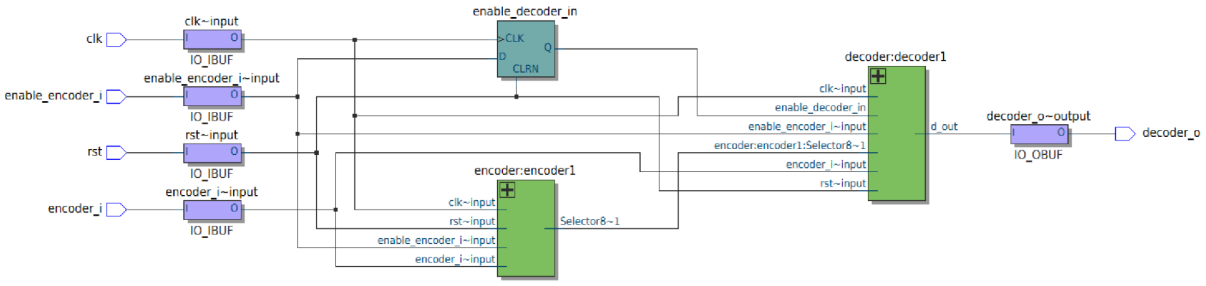

encoder:encoder1
valid_o
d_out~[1..0]
d_out[1..
2'h0
0
1
WideNor0
Selector8
SEL[7..0]
DATA[7..0]
OUT
cstate
clk
d_in
enable_i
reset
000
001
010
011
100
101
110
1'h0
cstate~9

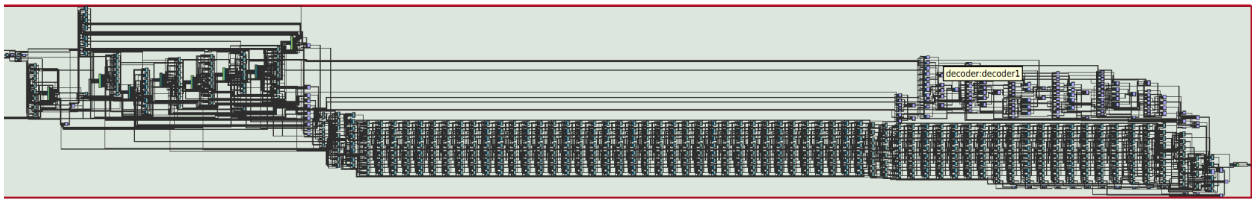


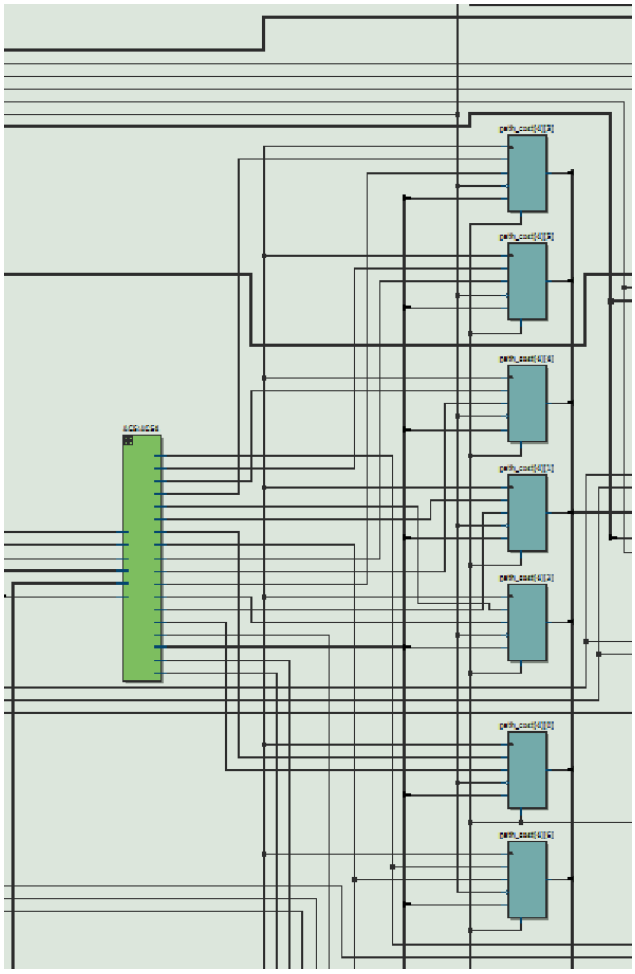



Mux1
Mux2
Mux3
Mux4
Mux5
Mux6
Mux7
Mux8
Mux0
Add0
Add1
LessThan0
Decoder0
path_cost~[7..0]
valid_o~not


bmc:bmc4_inst
rx_pair[1..0]
path_0_bmc~0
path_0_bmc~1
path_1_bmc~0
path_1_bmc~1
path_0_bmc[1..0]
path_1_bmc[1..0]

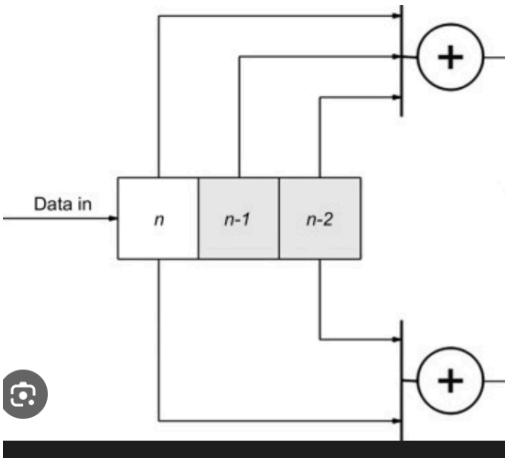






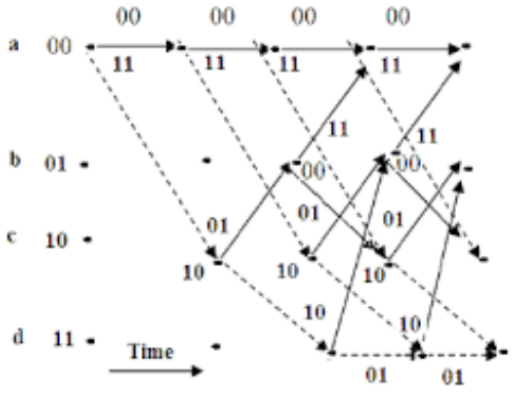
The post mapping schematic shows just how crazy the wires can look when the decoder has to go through each possible path through the trellis. Above, we can see one of the ACS units running and ‘storing’ calculations for the paths from the BMC component.

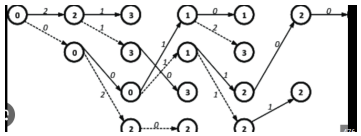
Encoder:

- The encoder was given as a transition diagram in the instructions. Looking at the table, we can see it is a rate 1/2, constraint length K=3 convolutional encoder. For every 1 input bit, it produces 2 output bits, and the output depends on the current input plus the last 2 bits of history (hence constraint length 3, giving $2^3 = 8$ states).
- To find the pattern, the key is to look at what determines d_out_reg. The output only ever takes 4 values, then just group the states by output. The groups emerge as 2 distinct groups of states, where the state's output is clearly dependent on the XOR output of the state bits. The xor dependencies come out to out[1] = s2 XOR s0 out[0] = s2 XOR s1 XOR s0 (where s2 is the input bit). The structure sounds quite familiar and is very similar to the encoder in class. Here is a diagram I found from a Google search that resembles the structure:
  - 

- In my actual implementation code, I hardcoded the states because I had the table as a specification and wanted to make sure there were no errors in the modules that shouldn't take much time, so I created case statements for transitions based on the table as a 1:1 model of the specification given. Once I tested it and knew it worked, I then focused on the more time-intensive part.

Decoder:

- The Viterbi decoder works by selecting the most likely path (which it essentially finds by minimizing a loss function - ikr, did not think I would make a connection to machine learning here) in the trellis. Here is an example of a trellis diagram:
  - 

- To compute the paths and the loss function, the decoder checks every possible state and computes the branch metric
    - This part is handled by the BMC
- BMC - The BMC computes the branch metrics, or how “far” a given state is to the correct state or the number of bits that are different.
    - 
    - My BMC looks something like this:
    - ```path_0_bmc[1] = tmp00 & tmp01; // 2-bit hamming: both wrong = distance 2 path_0_bmc[0] = tmp00 ^ tmp01; // one wrong = distance 1``` Which explains how it creates a score by creating a binary value of “loss” or the value of the branch metric (how many bits are off) which means the branc metric is 0 when there are no wrong bits.
- The metrics then get added to paths, and since each state can get reached by exactly 2 previous states, our path can go different routes. This is why the next step is the ACS, the Add Compare Select module that adds up all the metrics in a path to choose the optimal route (this is essentially Dijkstra’s, but for decoding, since we are choosing the path with the lowest weights).
    - The ACS compares the cost of the two previous states and chooses the cheaper one. The selection bit records which predecessor won, an important piece of information saved for traceback.
- The survivor history buffer contains a record of which predecessor was chosen at every state at every timestep. The decoder first finds the state with the lowest total path cost (best_state), then reads back 64 steps through the survivor_hist shift register, and finally, the oldest bit in that history is the decoded output.
- Finally, the input bit corresponding to the better path (lowest metric) gets appended, and the output is assigned.

Part 2

The module was *parameterized* so that I could just change the inputs to the module in the testbench easily, instead of rewriting the module and taking every case into account.

A

Example outputs (all relevant data recorded in spreadsheet):

```
# error_counter,err_inj = 0000001c 00          28         223
# error_counter,err_inj = 0000001c 00          28         224
# error_counter,err_inj = 0000001d 01          28         225
# error_counter,err_inj = 0000001d 00          29         226
# error_counter,err_inj = 0000001d 00          29         227
# error_counter,err_inj = 0000001d 00          29         228
# error_counter,err_inj = 0000001d 00          29         229
# error_counter,err_inj = 0000001d 00          29         230
# error_counter,err_inj = 0000001d 00          29         231
# error_counter,err_inj = 0000001d 00          29         232
# error_counter,err_inj = 0000001e 01          29         233
# error_counter,err_inj = 0000001e 00          30         234
# error_counter,err_inj = 0000001e 00          30         235
# error_counter,err_inj = 0000001e 00          30         236
# error_counter,err_inj = 0000001e 00          30         237
# error_counter,err_inj = 0000001e 00          30         238
# error_counter,err_inj = 0000001e 00          30         239
# error_counter,err_inj = 0000001e 00          30         240
# error_counter,err_inj = 0000001f 01          30         241
# error_counter,err_inj = 0000001f 00          31         242
# error_counter,err_inj = 0000001f 00          31         243
```

In order (1-8):

```
# yaa! in = 1, out = 1, w_ct =         253, err = 00
# yaa! in = 1, out = 1, w_ct =         254, err = 00
# yaa! in = 1, out = 1, w_ct =         255, err = 00
# corrupted_bits =          32, OUT: good =         256, bad =           0
# corrupted_bits =          32, OUT: good =         256, bad =           0
# yaa! in = 1, out = 1, w_ct =         255, err = 00
# corrupted_bits =          16, OUT: good =         256, bad =           0
# corrupted_bits =          32, OUT: good =         256, bad =           0
# corrupted_bits =          32, OUT: good =         256, bad =           0
# corrupted_bits =          32, OUT: good =         217, bad =          39
# corrupted_bits =          32, OUT: good =         241, bad =          15
# corrupted_bits =          16, OUT: good =         232, bad =          24
```

B

(table includes part a)

test #	spacing	interval	corrupt	repeat		corrupt	good	bad
2a1	even	8	[0]	1		32	256	0
2a2	even	8	[1]	1		32	256	0
2a3	even	16	[1:0]	1		16	256	0
2a4	even	16	[0]	2		32	256	0
2a5	even	16	[1]	2		32	256	0
2a6	even	32	[0]	4		32	217	39
2a7	even	32	[1]	4		32	241	15
2a8	even	32	[1:0]	2		16	232	24
2b1	random	8		1		135	122	134
2b2	random	8		1		135	214	42
2b3	random	16		1		123	139	117
2b4	random	16		2		128	126	130
2b5	random	16		2		128	209	47
2b6	random	32		4		128	131	125
2b7	random	32		4		128	216	40
2b8	random	32		2		126	134	122

In order (1-8):

```
 # corrupted_bits =         135, OUT: good =         122, bad =          134
 # corrupted_bits =         135, OUT: good =         214, bad =           42
# corrupted_bits =          123, OUT: good =         139, bad =          117
 # corrupted_bits =         128, OUT: good =         126, bad =          130
# corrupted_bits =          128, OUT: good =         209, bad =          47
 # corrupted_bits =         128, OUT: good =         131, bad =          125
 # corrupted_bits =         128, OUT: good =         216, bad =          40
 # corrupted_bits =         126, OUT: good =         134, bad =          122
```

C, D, E

Minimum consecutive bad bits to produce output errors					
2C	[0]	5 bits		4 errors	
2D	[1]	5 bits		4 errors	
2E	[1:0]	5 bits		4 errors	

All cases found the same amount of repeat error bits to cause an output error: 5 bits. I used an interval of 256 to get the injected errors once.
The Viterbi decoder can guarantee correction of any single burst of errors affecting fewer than 5 coded symbols. Five corrupted bits in one lane are enough to push the path metric difference beyond what the decoder can reliably resolve.

```
# corrupted_bits =          5, OUT: good =         252, bad =           4
# corrupted_bits =          5, OUT: good =         252, bad =           4
# corrupted_bits =          5, OUT: good =         252, bad =           4
```